

Contents

1 Chats

Exported on: 1/17/2026, 11:28:42 AM

1.1 You

YouTube Analyzer!

I am planning an Google Chrome extension that analyze the Youtube browser page with focus on the proposed videos. The proposed videos should ranked and attributed into several categories related to their production quality and production category or style.

There no judgment intended, but only analysis for a broader view.

Please search for similar ideas and existing code. The most important parts are algorithms that decide the category assignment. I knew it would possible to train an LLM to do this job, but the training would contain human judgement injecting this into the weights. I am searching for real entropy related data. For example a still image zoom slowly contains no new information per frame, while a moving camera angle show new content, but compared to drone image from flying drone the information density is very different. To apply this to actual topic, like animated speakers and generated voices would be interesting also. Please search deep for ideas, algorithmhs, code and existing project to cooperate with.

1.2 Gemini

Show thinking

This is a scientifically grounded approach. You are essentially trying to build a “Signal Complexity Filter” rather than a semantic content filter. By bypassing LLMs and using raw information density (entropy), you avoid the “black box” of moral/cultural bias and focus on production effort and style.

Here is a breakdown of existing algorithms, the “Entropy” logic applied to video, and technical paths for a Chrome Extension.

1. The Core Algorithm: “Entropy” in Video

Your intuition about “zoom vs. drone” maps directly to two industry-standard metrics used in Video Quality Assessment (VQA): SI (Spatial Information) and TI (Temporal Information).

SI (Spatial Information): Measures the amount of detail in a single frame (Entropy of the image).

Low: Flat graphics, whiteboards, simple cartoons.

High: Drone footage of forests, complex sets, detailed crowds.

Algorithm: Apply a Sobel Filter (edge detection) to a frame and calculate the Standard Deviation of the pixels.

TI (Temporal Information): Measures the change between consecutive frames (Entropy of motion).

Low: Static zoom, slow pan, slideshows.

High: Action cameras, fast cuts, handheld vlogs.

Algorithm: Calculate the pixel-wise difference between $Frame_t$ and $Frame_{t-1}$ and take the Standard Deviation.

The “Real Entropy” Matrix for Categorization: You can plot every video on a 2D graph (SI vs. TI) to categorize them without AI:

StyleSI (Detail)TI (Motion)Entropy SignaturePodcast / Static AudioLowNear ZeroMinimal new info per second.Slideshow / TutorialHighLowBursts of info (slide change) then static.Talking HeadMedLow/MedLocalized motion (mouth/hands), static background.Gaming / VlogsMed-HighChaotic motion, often lower texture quality.Cinematic / DroneHighHighHigh texture + Global motion (entire frame moves).

Export to Sheets

2. Implementation Strategy for Chrome

Analyzing the full video stream for every recommendation is impossible (bandwidth/CPU suicide). You must use Proxies.

A. The “Storyboard” Hack (Best Performance)

YouTube fetches a “storyboard” (a sprite sheet of low-res images) for the progress bar hover preview.

Method: Intercept the request for the storyboard image (usually named M\$number in the network tab).

Analysis: It gives you a 10x10 grid of the whole video.

Color Histogram Entropy: If the color palette shifts wildly between shots, it’s a vlog/high-production video. If it stays mono-tone, it’s a podcast/essay.

Cut Detection: Count how many “blocks” in the sprite sheet are radically different. Low count = Long takes (Podcast). High count = Fast editing (MrBeast style).

B. The “Hover Preview” Analysis (Higher Fidelity)

When you hover over a thumbnail, YouTube plays a short WebP or MP4 snippet.

Method: Use an offscreen HTML5 Canvas to grab pixel data from this snippet.

Compression Ratio as Entropy: This is your “real entropy” data.

Read the Content-Length (file size) of the hover snippet.

Divide by the duration (e.g., 3 seconds).

Logic: A 3-second clip of a static image zoom will be tiny (50KB). A 3-second clip of a drone flight will be huge (500KB) because the encoder cannot compress the changing pixels. File size is entropy.

3. Detecting “Fake” Humans (AI Avatars)

You mentioned animated speakers. “Real” entropy helps here too.

Micro-Motion Analysis: Real humans have “involuntary jitter” (breathing, micro-expressions). AI avatars (like HeyGen) are often perfectly still outside the mouth/eyes.

Audio Spectrogram Entropy:

Real Voice: High dynamic range, imperfect frequencies, background noise floor (high entropy).

AI Voice: Perfectly clean frequencies, “dead” silence between words (mathematically low entropy).

Tools: You can use Web Audio API in Chrome to get the frequency data of the preview audio.

4. Existing Code & Libraries

Python (for prototyping):

VCA (Video Complexity Analyzer): A research project specifically checking spatial/temporal complexity.

Search Term: VCA video complexity analyzer github

SI/TI Calculation:

Repository: NabajeetBarman/SI-TI (GitHub). Contains standard MATLAB/Python implementations of the ITU-T P.910 standard.

Scikit-Video: Has built-in motion estimation tools.

JavaScript (for the Extension):

FFmpeg.wasm: You can run FFmpeg inside the browser. It allows you to run filters like frozenframes (detects lack of motion) or blackframe on the snippets.

Note: Heavy for a background extension, use sparingly.

Optical Flow in JS:

Library: js-feat or tracking.js.

Use: Calculate “Global Motion” vectors. If all vectors move right = Pan. If vectors radiate out = Zoom. If vectors are random = Action.

5. Recommended Next Step

Do not try to build the classifier immediately. Build the “Entropy Meter” first. Create a simple script that overlays the Bitrate-per-pixel ($\text{File Size} / (\text{Resolution} * \text{Time})$) on top of YouTube thumbnails. You will likely see immediate clusters:

< 0.1 bits/pixel: Static/AI generated/Slides.

0.5 bits/pixel: High complexity/Camera movement.

This video covers the distinction between “fake” and “real” video data using Python analysis, touching on the “micro-expressions” (entropy of emotion) that you can use to distinguish AI avatars from real humans.

... Detecting AI generated Videos in Python ...

The video is relevant because it demonstrates how to use code (Python/OpenCV) to analyze frame-by-frame data (emotion/confidence variance) to detect “synthetic” content, which directly aligns with your goal of identifying generated voices/avatars via entropy-like variance metrics.

Detecting AI generated Videos in Python: Fake vs Real (DeepFake Detector) - YouTube

